Le jeu de Scrabble

Cahier des charges

9 janvier 2025
Version 0.1

Ce projet a pour objectif de développer un logiciel permettant de jouer au Scrabble de 2 à 4 joueurs. Le programme doit permettre aux joueurs de s'affronter sur un plateau de jeu en respectant les règles.

Mots-clefs: Deux à quatre joueurs, stochastique, information partielle.

1 Généralités

F1. Langage de programmation

Le langage de programmation de ce projet doit être choisi parmi les langages suivants :

- **Langage C** : C11 (ISO/IEC 9899 :2011).
- **Langage Java** : Java SE 17+.
- **Langage Python** : Python 3.7+.

F2. Style de codage

Le *coding style* du programme doit respecter :

- **Langage C** : le *coding style* de GNU.
- **Langage Java** : le *coding style* de Google.
- **Langage Python** : le *coding style* du PEP8.

F3. Langue par défaut dans le code

La documentation, le nom des variables, des fonctions et des fichiers devront être en anglais.

F4. Système cible

Le programme devra fonctionner sur des systèmes d'exploitation GNU/Linux.

F5. Documentation

Le programme et le code source devra être documenté : manuel utilisateur, option d'aide en ligne de commande et commentaires dans le code source.

F6. Tests

Le programme devra être testé et validé par suffisamment de tests pour garantir son bon fonctionnement (couverture d'au moins 80%). Les tests doivent être automatisés et facile à lancer.

F7. Bugs

Aucun bug menant à un crash ne doit être présent dans le programme. Les spécifications doivent être respectées et les erreurs doivent générer des messages d'erreur.

F8. Performances

Les performances du programme doivent être mesurées et documentées pour différentes entrées du programme ainsi que pour ses cas limites.

2 Bibliothèques et outils

F9. Build-system

Le '*build-system*' devra être basé sur :

- **Langage C** : `CMake`.
- **Langage Java** : `Maven`.
- **Langage Python** : `pyproject.toml + setuptools`.

F10. Frameworks de tests

Le *framework* de test sera basé sur :

- **Langage C** : `CTest + googletest + gcovr`.
- **Langage Java** : `JUnit + Jacoco`.
- **Langage Python** : `pytest + coverage`.

F11. Gestion des options

Les options en ligne de commande seront gérées par :

- **Langage C** : le module `getopt_long`.
- **Langage Java** : la bibliothèque `commons-cli`.
- **Langage Python** : le module `argparse`.

F12. Bibliothèque graphique

La bibliothèque graphique utilisé sera basé sur :

- **Langage C** : LibGtk 4.x.
- **Langage Java** : JavaFx.
- **Langage Python** : PyGObject.

F13. Internationalisation

Le *framework* d'internationalisation sera basé sur :

- **Langage C** : la bibliothèque `gettext`.
- **Langage Java** : `ResourcesBundle` et `Locale`.
- **Langage Python** : le module Python `gettext`.

La langue par défaut, sans locale précisée, sera l'anglais et, au moins, le français sera ajouté.

3 Options en ligne de commande

F14. Nom de l'exécutable principal

L'exécutable principal doit s'appeler '`scrabble`' et permettre de lancer tout le programme.

F15. Usage général

Le programme sera lancé comme ceci :

```
#> scrabble [OPTIONS] [FILENAME]
```

Si une option est incorrecte, le programme devra afficher un message d'erreur et l'aide du programme.

Si un nom de fichier est donné, le programme devra charger une partie depuis ce fichier. Sinon, le programme devra démarrer une nouvelle partie.

Les messages à l'utilisateur seront écrits sur la sortie standard (`stdout`) et les messages d'erreurs devront être écrits sur la sortie d'erreur standard (`stderr`). De plus, le code de retour du programme devra être `EXIT_SUCCESS` en cas de succès et `EXIT_FAILURE` en cas d'erreur.

F16. Aide en ligne de commande

L'option '`-h`', '`--help`' affiche l'aide puis quitte.

F17. Version

L'option '`-V`', '`--version`' affiche la version puis quitte.

F18. Mode `verbose`

L'option '`-v`', '`--verbose`' permettra d'afficher des informations supplémentaires pendant l'exécution.

F19. Mode `debug`

L'option '`-d`', '`--debug`' permettra d'afficher des informations de débogage pendant l'exécution.

F20. Mode `blitz`

L'option '`-b`', '`--blitz`' permettra de jouer en mode blitz. Dans ce mode, un temps limite sera donné à chaque joueur pour jouer un coup. Si le temps d'un des joueur est écoulé, il perd la partie.

F21. Durée du blitz

L'option '`-t TIME`', '`--time TIME`' permettra de spécifier le temps limite en minutes pour chaque joueur en mode blitz. Par défaut, le temps limite sera de 30 minutes par joueur. Si cette option est donnée sans l'option '`-b`', '`--blitz`', elle sera ignorée mais un message d'avertissement sera affiché.

F22. Nombre de joueurs

L'option '`-p N`', '`--players N`' permettra de spécifier le nombre de joueurs pour une partie (de 2 à 4). Par défaut, le jeu se jouera à deux joueurs.

F23. Langue du jeu

L'option '`-l LANG`', '`--language LANG`' permettra de spécifier la langue du jeu. Par défaut, le jeu sera en anglais (`en`). Les langues supportées seront au moins : anglais (`en`), français (`fr`). Il devra être possible de lister les langues supportées avec l'option '`--list-languages`'.

F24. Mode Super-scrabble

L'option '`-s`', '`--super`' permettra de jouer en mode Super-scrabble. Dans ce mode, le plateau de jeu sera plus grand (21 par 21) et le nombre de lettres est augmenté.

F25. Dictionnaire personnalisé

L'option '`-D FILENAME`', '`--dictionary FILENAME`' permettra de spécifier un fichier de dictionnaire personnalisé. Par défaut, le programme utilisera le dictionnaire fourni avec le programme.

F26. Mode `contest`

L'option '`-c FILENAME`', '`--contest FILENAME`' permet de lancer le programme en mode '`contest`' en chargeant une partie depuis un fichier '`FILENAME`'. Le programme devra alors afficher le joueur et le coup à jouer au format décrit dans cette spécification.

F27. Mode IA

L'option '`-a[PLAYER]`', '`--ai[=PLAYER]`' permet de lancer le programme en mode joueur artificiel pour le joueur `PLAYER` ('1', '2', ...et '`A`' (*All*), '2' par défaut).

4 Interface utilisateur

F28. Interface en ligne de commande

Le programme devra posséder une interface en ligne de commande qui permette une interaction avec l'utilisateur via le terminal. Cette interface textuelle sera lancée par défaut par le programme.

F29. Interface graphique

Le programme devra posséder une interface graphique qui permette les mêmes interactions que sur le CLI avec l'utilisateur via une fenêtre graphique. Cette interface graphique sera lancée si l'option '`-g`', '`--gui`' est donnée.

F30. Affichage du plateau

À chaque coup pour un humain, le programme devra afficher le plateau ainsi que le temps pris par chaque joueur si la partie est en blitz.

En mode ASCII les cases vides seront représentées par un '_' et les lettres par leur lettre majuscule. Les cases spéciales seront mises en exergue par des couleurs différentes en utilisant les codes ANSI pour changer la couleur de background. Les cases 'lettre compte double' seront cyans, les cases 'lettre compte triple' seront bleues, les cases 'mot compte double' seront rouge et les cases 'mot compte triple' seront violettes.

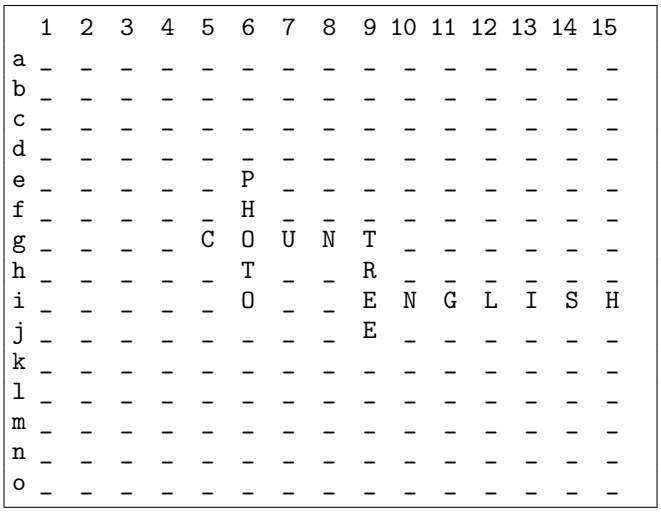
Dans le cas du Super-scrabble, les cases 'lettre compte double' seront cyans léger (*light cyan*), 'lettre compte triple' seront cyans, les cases 'lettre compte quadruple' seront violettes, les cases 'mot compte double' seront rouges léger (*light red*), les cases 'mot compte triple' seront rouges et les cases 'mot compte quadruple' seront violettes.

F31. Nommer les joueurs

Les utilisateurs pourront remplacer leur numéro de joueur par un nom à tout moment de la partie. C'est ce nom qui sera affiché à l'écran pour indiquer le joueur courant et les scores.

F32. Placement d'un mot sur le plateau

Le programme devra permettre de placer un mot composé à partir de ses lettres sur le plateau. Les cases seront numérotées comme indiquées dans la figure suivante :



Le joueur devra entrer les coordonnées de la case de départ, la direction du mot (horizontale ou verticale) et le mot à placer. Par exemple, pour placer le mot 'COUNT' en partant de la case g5 en direction verticale, le joueur entrera : '`g5v COUNT`'. Le programme devra vérifier que le mot peut effectivement être composé à partir de la réserve de lettres du joueur, que les cases sont bien vides là où il faut et qu'il est bien présent dans le dictionnaire de la partie. Si la proposition du joueur est incorrecte, un message d'erreur qui indiquera la cause du problème devra être affiché et le programme devra demander une autre action.

F33. Passer son tour

Le programme devra permettre à un joueur de passer son tour s'il ne peut pas ou ne veut pas jouer de mot.

F34. Échange de lettres

Le programme devra permettre à un joueur d'échanger un certain nombre de lettres de sa réserve contre un nombre équivalent de lettres prises dans le sac. Le joueur devra entrer les lettres à échanger. Par exemple, pour échanger les lettres 'A', 'B' et 'C', le joueur entrera : '`ABC`'. Le programme devra vérifier que les lettres sont bien présentes dans la réserve du joueur et qu'il reste suffisamment de lettres dans le sac. Si la proposition du joueur est incorrecte, un message d'erreur qui indiquera la cause du problème devra être affiché et le programme devra demander une autre action.

F35. Gestion du sac de lettres

Le programme devra gérer un sac de lettres qui contiendra toutes les lettres du jeu. À chaque fois qu'un joueur pioche des lettres, elles devront être retirées du sac. Si le sac est vide, le programme devra afficher un message et la partie sera terminée.

F36. Abandon

Le programme devra permettre à un joueur d'abandonner la partie en cours.

F37. Comptage et affichage des scores

Le programme devra afficher les scores des joueurs à chaque coup. Les scores attribués à chaque lettre dépendent de la langue utilisée, le programme devra être capable de gérer plusieurs langues et donc plusieurs valeurs de lettres. Par défaut, le programme utilisera les valeurs des lettres qui correspondent à la locale de l'utilisateur qui lance le programme ou bien l'anglais si la langue n'est pas supportée.

F38. Quitter et sauvegarde une partie

Le programme devra permettre de quitter une partie en cours et de la sauvegarder dans un fichier.

F39. Sauvegarde de l'historique

Le programme devra pouvoir sauvegarder l'historique des coups joués dans un fichier à la demande de l'utilisateur.

F40. Naviguer dans l'historique

Il doit être possible de revenir en arrière ou d'aller en avant dans l'historique des coups joués si tous les joueurs humains sont d'accord.

F41. Recommencer une partie

À tout moment, il doit être possible de recommencer une partie depuis le début en abandonnant la partie en cours.

F42. Rejouer une partie

Il doit être possible de rejouer une partie depuis le début en chargeant un fichier d'historique. Il doit être possible d'avancer et de reculer dans l'historique des coups voire de recommencer une partie depuis une position choisie par l'utilisateur (mais l'historique de la partie initiale au-delà de la position choisie par l'utilisateur sera perdu).

Par défaut, le programme affichera initialement l'état qui se situe à la fin de l'historique après son chargement.

5 Formats d'entrée/sortie

F43. Format de fichier simplifié

Le programme devra être capable de sauvegarder et restaurer des fichiers dans un format ASCII simplifié qui représente un goban de taille quelconque. Par exemple, un fichier de sauvegarde pourrait ressembler à ceci :

```
# Commentaire
1 # joueur courant
# Les scores des joueurs
1 15
2 10
# Puis le plateau de jeu
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _
_  _  _  _  _  P  _  _  _  _  _  _  _  _  _
_  _  _  _  _  H  _  _  _  _  _  _  _  _  _
_  _  _  _  C  O  U  N  T  _  _  _  _  _  _
_  _  _  _  _  T  _  _  R  _  _  _  _  _  _
_  _  _  _  _  O  _  _  E  N  G  L  I  S  H
_  _  _  _  _  _  _  _  E  _  _  _  _  _  _
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _
# En option, l'historique des coups joués
1. 1 g5h COUNT 2 e6v PHOTO
...
```

Une erreur de lecture dans un fichier devra être signalée par une erreur à l'utilisateur en spécifiant le type d'erreur et la ligne où elle a été trouvée et le programme devra s'arrêter.

F44. Format de l'historique

Le programme devra être capable de lire et d'écrire des historiques de coups joués avec des commentaires marqués par les caractères '`{}`'. Ils se présenteront comme une liste de mots placés par les joueurs, par exemple :

```
# Commentaire
# Coups joués
1. 1 g5h COUNT {12 pts} 2 e6v PHOTO {15 pts}
2. 1 g9v TREE {10 pts} 2 i9h ENGLISH {20 pts}
...
```

F45. Fichier de configuration

Le programme devra être capable de lire un fichier de configuration (`.scrabblerc`) qui contiendra les options par défaut du programme. Ce fichier sera lu au démarrage du programme et les options qu'il contient seront utilisées par défaut. Les options passées en ligne de commande devront écraser les options du fichier de configuration. On utilisera le format INI pour ce fichier, par exemple :

```
[Default]
blitz = false
time = 30
ai = None
```

6 Structure internes

F46. Dictionnaire

Le programme devra comporter un module `lexicon` qui permettra de charger un dictionnaire de mots. Ce dictionnaire sera utilisé pour vérifier si un mot est valide. Le format d'entrée du dictionnaire sera un fichier texte contenant un mot par ligne. Le programme devra charger ce dictionnaire au démarrage en fonction de la langue choisie par l'utilisateur. Par défaut, le programme chargera le dictionnaire qui correspond à la locale de l'utilisateur s'il possède un dictionnaire pour cette langue, sinon il chargera le dictionnaire anglais.

F47. Module `gaddag`

Le module `gaddag` permettra de gérer un arbre de recherche de mots (Dawg) qui permettra de trouver rapidement les mots possibles à partir d'un ensemble de lettres. On utilisera les techniques décrites dans les articles [1, 2] pour gérer cet arbre.

7 Interface graphique

F48. Fonctions de base

Toutes les fonctions demandées pour l'interface en console devront être implémentées dans l'interface graphique.

F49. Menu '`File`'

L'interface graphique devra comporter un menu principal qui permettra de lancer une nouvelle partie ('`New Game`'), de charger une partie depuis un fichier ('`Load Game`'), de sauvegarder la partie en cours dans un fichier ('`Save Game`'), de quitter le programme ('`Quit`'), d'afficher l'aide ('`Help`') ou d'éditer la configuration par défaut ('`Settings`').

F50. Menu '`Game`'

L'interface graphique devra comporter un menu de jeu qui permettra de demander un conseil ('`Hint`'), naviguer dans l'historique (revenir en arrière, aller en avant, etc).

F51. Menu '`About`'

L'interface graphique devra comporter un menu d'options qui permettra de consulter des informations sur le programme (version, auteurs, ...).

F52. Affichage d'une partie en cours

L'interface graphique devra afficher le goban de la partie en cours ainsi que les informations habituellement présentes sur l'interface en ligne de commandes.

F53. Affichage des coups joués

L'interface graphique devra afficher sur un côté de l'interface, les coups joués lors de la partie en les numérotant.

F54. Configuration d'une nouvelle partie

L'item '`New Game`' du menu '`File`' permettra de mener l'utilisateur à une fenêtre de configuration de la partie qui sera déjà remplie avec les paramètres par défaut. Qu'ils soient modifiés ou non, l'utilisateur pourra lancer la partie en cliquant sur '`Start`'.

8 Joueur artificiel

F55. Recherche de mots possibles

Le programme devra être capable de trouver tous les mots possibles à partir des lettres d'un joueur et d'un plateau de jeu en se basant sur le module `gaddag`.

F56. Heuristique d'évaluation de position

Le programme devra comporter au moins une fonction capable de prendre un état du jeu et de lui donner un score en fonction de différents critères qui vous sembleront pertinents. Il devra évaluer les lettres posées et celles restantes, les cases spéciales, les mots déjà posés et leur occupation du plateau, etc. Ce score devra donner idée de la qualité de chaque coup qu'il peut jouer.

F57. Recherche du meilleur coup (Minimax)

Le choix du meilleur coup se fera via une exploration de l'arbre des coups possibles via un algorithme de recherche Minimax ou équivalent. Cette option ne sera pas activée par défaut si le programme est lancé en mode joueur artificiel. L'option '`--ai-mode minimax`' sera disponible pour l'activer. Si cette option n'est pas activée, le programme proposera le meilleur coup trouvé pour ce tour sans anticiper les coups des autres joueurs.

F58. Recherche du meilleur coup (Monte-Carlo Tree Search)

Le choix du meilleur coup se fera via une exploration de l'arbre des coups possibles via l'algorithme de *Monte-Carlo Tree Search*. Cette option ne sera pas activée par défaut si le programme est lancé en mode joueur artificiel. Néanmoins, l'option '`--ai-mode mcts`' sera disponible pour l'activer. Si cette option n'est pas activée, le programme proposera le meilleur coup trouvé pour ce tour sans anticiper les coups des autres joueurs.

F59. Profondeur de recherche

L'option '`--ai-depth DEPTH`' permettra de spécifier la profondeur de recherche de l'algorithme de recherche.

F60. Temps de réflexion borné

L'option '`--ai-time TIME`' permettra de spécifier le temps de réflexion de l'algorithme de recherche. Si le temps est écoulé, le programme devra jouer le meilleur coup trouvé jusqu'à présent. Par défaut, le temps de réflexion sera de 5 secondes.

Références

[1] Andrew W. Appel and Guy J. Jacobson. The world's fastest scrabble program. *Communications of the ACM*, 31(5) :572–578, 1988.

[2] Steven A. Gordon. A faster scrabble move generation algorithm. *Journal of Software : Practice and Experience, Wiley*, 24(2) :219–232, 1994.